

Communication-Efficient Non-Interactive Proofs of Knowledge with Online Extractors

Marc Fischlin*

Institute for Theoretical Computer Science,
ETH Zürich, Switzerland

`marc.fischlin@inf.ethz.ch`
`http://www.fischlin.de/`

Abstract We show how to turn three-move proofs of knowledge into non-interactive ones in the random oracle model. Unlike the classical Fiat-Shamir transformation our solution supports an online extractor which outputs the witness from such a non-interactive proof instantaneously, without having to rewind or fork. Additionally, the communication complexity of our solution is significantly lower than for previous proofs with online extractors. We furthermore give a superlogarithmic lower bound on the number of hash function evaluations for such online extractable proofs, matching the number in our construction, and we also show how to enhance security of the group signature scheme suggested recently by Boneh, Boyen and Shacham with our construction.

1 Introduction

The Fiat-Shamir transformation [FS86] is a well-known technique to remove interaction from proofs of knowledge and to derive signature schemes from such proofs. The starting point is a three-move proof between a prover, holding a witness w to a public value x , and a verifier. The prover sends a commitment com , then receives a random challenge ch from the verifier and finally replies with resp . For the non-interactive version the prover computes the challenge himself by applying a hash function H to the commitment. The security of this transformation has later been analyzed under the idealized assumption that the hash function behaves as a random oracle [BR93,PS00], and has led to security proofs for related signature schemes.

Limitations. In the interactive case, all common knowledge extractors work by repeatedly rewinding the prover to the step after having sent com and completing the executions with independent random challenges. This eventually yields two valid executions $(\text{com}, \text{ch}, \text{resp})$, $(\text{com}, \text{ch}', \text{resp}')$ for different challenges $\text{ch} \neq \text{ch}'$ from which the extractor can compute the witness w . The same technique is reflected in the security proofs of the non-interactive version: The extractor

* This work was supported by the Emmy Noether Programme Fi 940/1-1 of the German Research Foundation (DFG).

continuously rewinds to the point where the prover has asked the random oracle H about com and completes the executions with independent hash values to find two valid executions (called “forking” in [PS00]).

The notable fact above is that, although the proof is non-interactive, the extractor still works by rewinding. As pointed out by [SG02] for example, this causes problems for some cryptographic schemes. Consider for example the ElGamal encryption $(R, C) = (g^r, pk^r \cdot m)$ for messages m . One attempt to make this scheme secure against chosen-ciphertext attacks is to append a non-interactive proof of knowledge for $r = \log R$ to the ciphertext. The idea is that, giving such a proof, any party generating a ciphertext would already “know” r and therefore $m = C \cdot pk^{-r}$. In other words, decryption queries in a chosen-ciphertext attack should be simulatable with the help of the knowledge extractor. However, this intuition cannot be turned into a proof, at least not with the rewinding extractor. Consider for example an adversary which sequentially puts n hash queries for the proofs of knowledge and then asks a decryption oracle for ciphertexts involving these hash queries *in reverse order*. Then, to answer each decryption query the extractor would have to rewind to the corresponding hash query. By this, it destroys all previously simulated decryption queries and must redo them from scratch, and the overall running time would become exponential in n .

We remark that the rewinding strategy also causes a loose security reduction. The results in [PS00] show that, if the adversary makes Q queries to the random oracle and forges, say, Schnorr signatures in time T with probability ϵ , then we can compute discrete logarithms in expected time QT/ϵ with constant probability. Hence, the number of hash queries enters multiplicatively in the time/success ratio. In contrast, for RSA-PSS and similar schemes [Cor00, Cor02, KW03] tight reductions are known. For other schemes like discrete-logarithm signatures different approaches relying on potentially stronger assumptions have been taken to get tight security results [GJ03].

Constructing Online Extractors. The solution to the problems above is to use extractors which output the witness immediately, i.e., without having to rewind. Following the terminology of [SG02], where this problem was discussed but circumvented differently, we call them *online extractors*.¹ Informally, such an extractor is given the value x , a valid proof π produced by a prover and all hash queries and answers the prover made for generating this proof (i.e., even queries which are later ignored in the proof). The extractor then computes the witness w without further communication with the prover. Note that here we use the fact that we work in the random oracle model, where the extractor sees the hash queries.

One known possibility to build such online extractors is to use cut-and-choose techniques combined with hash trees [Pas03, Mer88]. That is, one limits the challenge space to logarithmically many bits and repeats the following atomic protocol sufficiently often in parallel. The prover computes the initial commitment com of the interactive protocols and computes the answers for all possible challenges.

¹ Sometimes such extractors are also called straight-line extractors in the literature.

Since there are only polynomially many challenges and answers, the prover can build a binary hash tree with all answers at the leaves. Then he computes the actual challenge as the hash value over $\mathbf{com}$ and the root of the tree, and opens only the corresponding answer and all siblings on the path up to the root as a proof of correctness. For reasonable parameters these revealed hash values easily add about 10,000 to 25,000 bits to the non-interactive proof for all executions together, and thus cause a significant communication overhead.

Here we propose a different approach to build online extractors, producing much shorter proofs than the tree-based solution while having comparable extraction error and running time characteristics. In this introduction we provide a simplified description of our solution, omitting some necessary modifications. We also start with a polynomially bounded challenge space and a non-constant number of parallel executions. For each execution i the prover first computes $\mathbf{com}_i$ but now tries all polynomially many challenges $\mathbf{ch}_i = 0, 1, 2, \dots$ and answers $\mathbf{resp}_i = \mathbf{resp}_i(\mathbf{ch}_i)$ till it finds one for which a predetermined (at most logarithmic) number of least significant bits of $H(x, \mathbf{com}, i, \mathbf{ch}_i, \mathbf{resp}_i)$ are all zero. The prover outputs the vector $(\mathbf{com}, \mathbf{ch}, \mathbf{resp})$, no further hash values need to be included, and the verifier now also checks in all executions that the lower bits of the hash values are zero.

The honest prover is able to find a convincing proof after a polynomial number of trials for each execution (except with negligible probability which can be adapted through parameters). It is also clear that any prover who probes at most one valid challenge-response pair for each execution most likely does not find a hash value with zero-bits.² If, on the other hand, the prover tries at least two samples, then the knowledge extractor can find them in the list of hash queries and compute the witness. It follows that the (online) extraction probability is negligibly close to the verifier’s acceptance probability.

Our construction, outlined above, still requires a non-constant number of parallel repetitions in order to decrease the soundness error from polynomial to negligible. However, for proofs which are already based on small challenges, such as RSA with small exponents or “more quantum-resistant” alternatives like the recently proposed lattice-based proofs with bit challenges [MV03], several repetitions have to be carried out anyway, and our construction only yields an insignificant overhead in such cases. For other scenarios, like proofs of knowledge for discrete logarithms, the repetitions may still be acceptable, e.g., if the proof is only executed occasionally as for key registration. Alternatively, for the discrete logarithm for example, the prover can precompute the commitments $\mathbf{com}_i = g^{r_i}$ offline and the verifier is able to use batch verification techniques [BGR98] to reduce the computational cost.

A Lower Bound. Both the hash-tree construction and our solution here require a non-constant number of repetitions of the atomic protocol. An interesting

² We presume that it is infeasible to find distinct responses to a single challenge. Indeed, this requirement is not necessary for the Fiat-Shamir transformation, yet all proofs we know of have this additional property.

question is if one can reduce this number. As a step towards disproving this we show that the number of hash function evaluations for the prover must be superlogarithmic in order to have an online extractor (unless finding witnesses is trivial, of course).³ While this superlogarithmic bound would be straightforward if we restrict the hash function’s output size to a few bits, our result holds independently of the length of hash values.

The proof of our lower bound requires that the knowledge extractor does not have the ability to choose the hash values. If we would allow the extractor to program the random oracle then we could apply the hash function to generate a common random string and run a non-interactive zero-knowledge proof of knowledge in the standard model (based on additional assumptions, though) [DP92]. For unrestricted (but polynomial) output length a single hash function evaluation for both the prover and verifier would then suffice. For non-programming extractors the number of hash function evaluations in our construction and the hash-tree solution are optimal with respect to general protocols.

A Word About Random Oracles. Our solution is given in the random oracle model, and a sequence of works [CGH98,GT03,MRH04,BBP04] has shown that constructions in this model may not yield a secure scheme in the real world when the oracle is instantiated by some function. It is therefore worthwhile to take a look at the way we utilize the random oracle. In our transformation we essentially use the random oracle as a predicate with the following properties: The only way to evaluate this predicate is by computing it explicitly (thus “knowing” the input), that predicate outcomes are well distributed (i.e., random), the predicate values for related inputs are uncorrelated.

In comparison to the Fiat-Shamir transformation our construction somewhat “decouples” the hash function from the protocol flow. Indeed, the dependency of the answer and the hash function in the Fiat-Shamir transformation is exploited by Goldwasser and Tauman [GT03] to prove insecurity of the random oracle approach for the transformation. Because of the aforementioned separation of the protocol flow and the hash function in our solution, the counterexample in [GT03] does not seem to carry over (yet, similar results may hold here as well). The point is that our solution is provided as an alternative to the Fiat-Shamir transformation, given one accepts the random oracle model as a viable way to design efficient non-interactive proofs. Finding truly efficient non-interactive proofs of knowledge without random oracles is still open.

Applications. Clearly, proofs of knowledge with online extractors are especially suitable for settings with concurrent executions such as key registration steps. As another, more specific example, we show that our method can be used to enhance security of the Boneh et al. group signature scheme [BBS04]. Roughly, a group

³ To be more precise, we give a slightly stronger result relating the number of hash queries of the verifier and the prover. This stronger result shows for example that hard relations cannot have efficient provers if the verifier only makes a constant number of hash function queries to verify proofs.

signature scheme allows a set of users to sign messages such that a signature does not reveal the actual signer, yet a group manager holding a secret information can revoke this anonymity and identify the true signer. A stringent formalization of these two properties, called full anonymity and full traceability, has been put forth by Bellare et al. [BMW03].

Although achieving strong traceability guarantees the protocol by Boneh et al. only realizes a slightly weaker anonymity notion. In the original definition [BMW03] anonymity of a signer of a message should hold even if an adversary can request the group manager to open identities for other signatures (thus resembling chosen-ciphertext attacks on encryption schemes). In the weaker model such open queries are not allowed, and this property is consequently called CPA-full-anonymity in [BBS04].

Without going into technical details we remark that the weaker anonymity property in [BBS04] originates from the underlying (variation of the) ElGamal encryption scheme and its CPA-security. A promising step towards fully anonymous group signature is therefore to turn the ElGamal encryption into a CCA-secure scheme. As explained before, standard Fiat-Shamir proofs of knowledge for the randomness used to generate ciphertexts do not work because of the rewinding problems. And although there is a very efficient method to secure basic ElGamal against chosen-ciphertexts in the random oracle model [Abe04], this technique inherently destroys the homomorphic properties of the ciphertexts. But this homomorphic property is crucial to the design of the group signature scheme as it allows to efficiently prove relations about the encrypted message.

Proofs of knowledge with online extractors provide a general solution. However, since one of the initial motivations of [BBS04] was to design a scheme with *short* signatures of a couple of hundred bits only, the aforementioned hash-tree based constructions with their significant communication overhead, for example, are prohibitively expensive. We show that with our protocol we obtain a fully-anonymous scheme and for reasonable parameters the length of signatures increases from 1,500 to about 5,000 bits. In comparison, the RSA-based group signature scheme in [ACJT00], presumably one of the most outstanding group signature schemes, still requires more than 10,000 bits. Based on implementation results about elliptic curves [DMPW98], and the fact that the signer in the scheme by Ataniese et al. [ACJT00] cannot apply Chinese-Remainder techniques to compute the exponentiations with 1,000 and more bits, we estimate that our variation of the Boneh group signature is still more efficient, despite the repetitions for the proof of knowledge. This is especially true for the verifier who can apply batch verification techniques on top.

Organization. In Section 2 we give the basic definitions of three-move Fiat-Shamir proofs of knowledge and non-interactive ones with online extractors in the random oracle model. The main part of the paper, Section 3, presents our construction, shows that it even achieves simulation-soundness and that this immediately gives secure signature schemes with tight security reductions. This section concludes with our lower bound on the number of hash queries. Section 4 finally presents our enhancement of the Boneh et al. group signature scheme.

2 Definitions

A security parameter k in our setting is an arbitrary string describing general parameters. In the most simple case $k = 1^\kappa$ describes the length κ of the cryptographic primitives in unary. More generally, k can for example consist of the description of a group $\mathcal{G}$ of prime order q and generator g of that group, i.e., $k = (\mathcal{G}, q, g)$. The security parameter also describes a sequence of efficiently verifiable relations $W = (W_k)_k$.

A (possibly negative) function $f(k)$ is called *negligible* if $f(k) \leq 1/p(k)$ for any polynomial $p(k)$ and all sufficiently large k 's. A function which is not negligible is called *noticeable*. For two functions f, g we denote by $f \gtrsim g$ the fact that $g - f$ is negligible. Accordingly, $f \approx g$ stands for $f \gtrsim g$ and $g \gtrsim f$. A function f with $f \gtrsim 1$ is called *overwhelming*.

We usually work in the random oracle model where parties have access to a random function H with some domain and range depending on k . We note that we do not let the relation W depend on the random oracle H in order to avoid “self-referencing” problems. We occasionally let an algorithm “output a random oracle”, $H \leftarrow A$, meaning that A generates a description of a (pseudo)random function H .

We require some additional properties of the underlying Fiat-Shamir proof to make our transformation work. First, we need that the prover's initial commitment `com` has nontrivial entropy. This can be achieved easily by appending a superlogarithmic number of public random bits to `com` if necessary. Second, we need that the verifier sends a uniform bit string as the challenge `ch`; all common proofs have this property. Third, we require that the prover's response is quasi unique, i.e., it should be infeasible to find another valid response `resp'` to a proof $(\text{com}, \text{ch}, \text{resp})$, even if one knows the witness. This holds for example if `resp` is uniquely determined by x, com, ch , e.g., as for the protocols by Guillou-Quisquater [GQ88] and Schnorr [Sch91], but also for Okamoto's witness-indistinguishable variations these protocols [Oka92] (if the parameter k contains the system parameters like the RSA modulus N with unknown factorization).

Definition 1. A Fiat-Shamir proof of knowledge (with $\ell(k)$ -bit challenges) for relation W is a pair (P, V) of probabilistic polynomial-time algorithms $P = (P_0, P_1)$, $V = (V_0, V_1)$ with the following properties.

[Completeness.] For any parameter k , any $(x, w) \in W_k$, any $(\text{com}, \text{ch}, \text{resp}) \leftarrow (P(x, w), V_0(x))$ it holds $V_1(x, \text{com}, \text{ch}, \text{resp}) = 1$.

[Commitment Entropy.] For parameter k , for any $(x, w) \in W_k$, the min-entropy of $\text{com} \leftarrow P_0(x, w)$ is superlogarithmic in k .

[Public Coin.] For any k , any $(x, w) \in W_k$ any $\text{com} \leftarrow P_0(x, w)$ the challenge $\text{ch} \leftarrow V_0(x, \text{com})$ is uniform on $\{0, 1\}^{\ell(k)}$.

[Unique Responses.] For any probabilistic polynomial-time algorithm A , for parameter k and $(x, \text{com}, \text{ch}, \text{resp}, \text{resp}') \leftarrow A(k)$ we have, as a function of k ,

$$\text{Prob}[V_1(x, \text{com}, \text{ch}, \text{resp}) = V_1(x, \text{com}, \text{ch}, \text{resp}') = 1 \wedge \text{resp} \neq \text{resp}'] \approx 0.$$

[*Special Soundness.*] There exists a probabilistic polynomial-time algorithm K , the knowledge extractor, such that for any k , any $(x, w) \in W_k$, any pairs $(\text{com}, \text{ch}, \text{resp})$, $(\text{com}, \text{ch}', \text{resp}')$ with $V_1(x, \text{com}, \text{ch}, \text{resp}) = V_1(x, \text{com}, \text{ch}', \text{resp}')$ $= 1$ and $\text{ch} \neq \text{ch}'$, for $w' \leftarrow K(x, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}')$ it holds $(x, w') \in W_k$.

[*Honest-Verifier Zero-Knowledge.*] There exists a probabilistic polynomial-time algorithm Z , the zero-knowledge simulator, such that for any pair of probabilistic polynomial-time algorithms $D = (D_0, D_1)$ the following distributions are computationally indistinguishable⁴:

- Let $(x, w, \delta) \leftarrow D_0(k)$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow (P(x, w), V_0(x))$ if $(x, w) \in W_k$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow \perp$ otherwise. Output $D_1(\text{com}, \text{ch}, \text{resp}, \delta)$.
- Let $(x, w, \delta) \leftarrow D_0(k)$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z(x, \text{YES})$ if $(x, w) \in W_k$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z(x, \text{NO})$ otherwise. Output $D_1(\text{com}, \text{ch}, \text{resp}, \delta)$.

Below we sometimes use a stronger kind of zero-knowledge property which basically says that the simulator is able to generate proofs for a specific challenge, as long as this challenge is given in advance. To formalize this let V_0^{CH} be a verifier which on input x, ch merely outputs ch . Then a Fiat-Shamir proof of knowledge (with $\ell(k)$ -bit challenges) is *special* zero-knowledge if the following holds:

[*Special Zero-Knowledge.*] There exists a probabilistic polynomial-time algorithm Z , the special zero-knowledge simulator, such that for any pair of probabilistic polynomial-time algorithms $D = (D_0, D_1)$ the following distributions are computationally indistinguishable:

- Let $(x, w, \text{ch}, \delta) \leftarrow D_0(k)$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow (P(x, w), V_0^{\text{CH}}(x, \text{ch}))$ if $(x, w) \in W_k$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow \perp$ else. Output $D_1(\text{com}, \text{ch}, \text{resp}, \delta)$.
- Let $(x, w, \text{ch}, \delta) \leftarrow D_0(k)$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z(x, \text{ch}, \text{YES})$ if $(x, w) \in W_k$, and $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z(x, \text{ch}, \text{NO})$ else. Output $D_1(\text{com}, \text{ch}, \text{resp}, \delta)$.

We note that all common protocols obey this special zero-knowledge property. In Appendix A we prove formally that *any* Fiat-Shamir proof of knowledge is special zero-knowledge if the challenge size $\ell(k) = O(\log k)$ is logarithmic (which holds for our transformation in the next section).

We next define **non-interactive proofs of knowledge with online extractors**. We note that, in the random oracle model, the verifier can be assumed wlog. to be deterministic. This is formally proven in Appendix B. The online extraction property says that the knowledge extractor K is able to output a witness w from any statement x with a valid proof π , given the hash queries the (possibly malicious) prover A made to prepare π . The extractor, unlike the zero-knowledge simulator, is not given the opportunity to program the random oracle, though.

Definition 2. A pair (P, V) of probabilistic polynomial-time algorithms is called a *non-interactive zero-knowledge proof of knowledge for relation W with an online extractor (in the random oracle model)* if the following holds.

⁴ Meaning that the probability that D_1 outputs 1 is the same in both experiments, up to a negligible difference.

[Completeness.] For any oracle H , any $(x, w) \in W_k$ and any $\pi \leftarrow P^H(x, w)$ we have $\text{Prob}[V^H(x, \pi) = 1] \gtrsim 1$.

[Zero-Knowledge.] There exist a pair of probabilistic polynomial-time algorithms $Z = (Z_0, Z_1)$, the zero-knowledge simulator, such that for any pair of probabilistic polynomial-time algorithms $D = (D_0, D_1)$ the following distributions are computationally indistinguishable⁵:

- Let H be a random oracle, $(x, w, \delta) \leftarrow D_0^H(k)$, and $\pi \leftarrow P^H(x, w)$ if $(x, w) \in W_k$, and $\pi \leftarrow \perp$ otherwise. Output $D_1^H(\pi, \delta)$.
- Let $(H_0, \sigma) \leftarrow Z_0(k)$, $(x, w, \delta) \leftarrow D_0^{H_0}(k)$, and $(H_1, \pi) \leftarrow Z_1(\sigma, x, \text{YES})$ if $(x, w) \in W_k$, and $(H_1, \pi) \leftarrow Z_1(\sigma, x, \text{NO})$ otherwise. Output $D_1^{H_1}(\pi, \delta)$.

[Online Extractor.] There exist a probabilistic polynomial-time algorithm K , the online extractor, such that the following holds for any algorithm A . Let H be a random oracle, $(x, \pi) \leftarrow A^H(k)$ and $\mathcal{Q}_H(A)$ be the sequence of queries of A to H and H 's answers. Let $w \leftarrow K(x, \pi, \mathcal{Q}_H(A))$. Then, as a function of k ,

$$\text{Prob}[(x, w) \notin W_k \wedge V^H(x, \pi) = 1] \approx 0.$$

Note that we allow the zero-knowledge simulator to program the random oracle, but only in two stages. Namely, Z_0 generates H_0 for D_0 and then Z_1 selects H_1 for the find-stage of D_1 . Since the adversary D_0 in the first stage can pass on all interactions with H_0 to D_1 through the state information δ , the simulator Z_1 must guarantee that H_1 is consistent with H_0 . However, Z_1 now has the opportunity to adapt oracle H_1 with respect to the adversarial chosen theorem x . Simulator Z_1 also gets the information whether x is in the language or not (in which case the simulator can simply set $\pi \leftarrow \perp$).

3 Constructions

Our starting point are interactive Fiat-Shamir proofs with logarithmic challenge length ℓ . Note that such proofs can be easily constructed from proofs with smaller challenge length l by combining $\lceil \ell/l \rceil$ parallel executions. It is easy to verify that all required properties are preserved by these parallel executions, including unique responses and honest-verifier zero-knowledge. Analogously, we can go the other direction and limit the challenge size to at most ℓ bits while conserving the properties.

3.1 Generic Construction

Recall the idea of our construction explained in the introduction. In each of the r repetitions we let the prover search through challenges and responses to find a tuple $(\text{com}, \text{ch}, \text{resp})$ whose b least significant bits of the hash are 0^b for a small b . From now on we assume for simplicity that H only has b output bits; this can always be achieved by cutting off the leading bits.

⁵ Meaning that the probability that D_1 outputs 1 is the same in both experiments, up to a negligible difference.

Instead of demanding that all r hash values equal 0^b we give the honest prover more flexibility and let the verifier accept also proofs $(\text{com}_i, \text{ch}_i, \text{resp}_i)_{i=1,2,\dots,r}$ such that the sum of the r hash values $H(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ (viewed as natural numbers) does not exceed some parameter S . With this we can bound the prover's number of trials in each execution by 2^t for another parameter t , slightly larger than b , and guarantee that the prover terminates in strict polynomial time.

For sake of concreteness the reader may think of $b = 9$ (output length of the hash function), $t = 12$ (challenge size), $r = 10$ (number of repetitions) and $S = 10 = r$ (maximum sum). For these values the probability of the honest prover failing to find a valid proof is about 2^{-60} , and the knowledge extractor will obtain the witness whenever the proof is valid, except with probability approximately $Q \cdot 2^{-70}$ where Q denotes the number of hash queries the prover makes.

Construction 1. *Let $(P_{\text{FS}}, V_{\text{FS}})$ be an interactive Fiat-Shamir proof of knowledge with challenges of $\ell = \ell(k) = O(\log(k))$ bits for relation W . Define the parameters b, r, S, t (as functions of k) for the number of test bits, repetitions, maximum sum and trial bits such that $br = \omega(\log k)$, $2^{t-b} = \omega(\log k)$, $b, r, t = O(\log k)$, $S = O(r)$ and $b \leq t \leq \ell$. Define the following non-interactive proof system for relation W in the random oracle model, where the random oracle maps to b bits.*

[Prover.] *The prover P^H on input (x, w) first runs the prover $P_{\text{FS}}(x, w)$ in r independent repetitions to obtain r commitments $\text{com}_1, \dots, \text{com}_r$. Let $\vec{\text{com}} = (\text{com}_1, \dots, \text{com}_r)$. Then P^H does the following, either sequentially or in parallel for each repetition i . For each $\text{ch}_i = 0, 1, 2, \dots, 2^t - 1$ (viewed as t -bit strings) it lets P_{FS} compute the final responses $\text{resp}_i = \text{resp}_i(\text{ch}_i)$ by rewinding, until it finds the first one such that $H(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i) = 0^b$; if no such tuple is found then P^H picks the first one for which the hash value is minimal among all 2^t hash values. The prover finally outputs $\pi = (\text{com}_i, \text{ch}_i, \text{resp}_i)_{i=1,2,\dots,r}$.*

[Verifier.] *The verifier V^H on input x and $\pi = (\text{com}_i, \text{ch}_i, \text{resp}_i)_{i=1,2,\dots,r}$ accepts if and only if $V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i, \text{resp}_i) = 1$ for each $i = 1, 2, \dots, r$, and if $\sum_{i=1}^r H(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i) \leq S$.*

Note that for common iterated hash functions like SHA-1 the prover and the verifier can store the intermediate hash value of the prefix $(x, \vec{\text{com}})$ and need not compute it from scratch for each of the r repetitions.

Our protocol has a small completeness error. For deterministic verifiers this error can be removed in principle by standard techniques, namely, by letting the prover check on behalf of the verifier that the proof is valid before outputting it; if not the prover simply sends the witness to the verifier. In practice, in case of this very unlikely event, the prover may just compute a proof from scratch.

Theorem 2. *Let $(P_{\text{FS}}, V_{\text{FS}})$ be an interactive Fiat-Shamir proof of knowledge for relation W . Then the scheme (P, V) in Construction 1 is a non-interactive zero-knowledge proof of knowledge for relation W (in the random oracle model) with an online extractor.*

Proof. We show that completeness, zero-knowledge and online extraction according to the definition are satisfied.

Completeness. For the completeness we show that the prover fails to convince the verifier with negligible probability only. For this let $\mathbf{s}_i$ be the random value $H(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ associated to the output of the i -th execution. Then,

$$\text{Prob}[\exists i : \mathbf{s}_i > S] \leq r \cdot (1 - (S+1)2^{-b})^{2^t} \leq r \cdot e^{-(S+1)2^{t-b}}$$

because in each of the at most 2^t tries the prover gets a random hash value of at most S with probability at least $(S+1)2^{-b}$, and all hash values are independent. The probability of having a value larger than S in one execution is thus negligible as r is logarithmic and 2^{t-b} is superlogarithmic. Hence, the sum of all r values exceeds rS with negligible probability only, and we from now on we can condition on the event that the sum of all $\mathbf{s}_i$ is at most rS . We also presume $r \geq 2$ in the sequel, else the claim already follows.

In order for the honest prover to fail the sum T of the r values $\mathbf{s}_1, \dots, \mathbf{s}_r \geq 0$ must be larger than S . For any such $T = S+1, S+2, \dots, rS$ there are at most $\binom{T+r-1}{r-1}$ ways to split the sum T into r non-negative integers $s_1, \dots, s_r$.⁶ This is upper bounded by

$$\binom{T+r-1}{r-1} \leq \left(\frac{e(rS+r-1)}{r-1} \right)^{r-1} \leq (e(2S+1))^{r-1} \leq e^{r \ln(e(2S+1))}$$

On the other hand, the probability of obtaining such a sum for a given partition, $\mathbf{s}_1 = s_1, \dots, \mathbf{s}_r = s_r$, is at most

$$\begin{aligned} \prod_{i=1}^r \text{Prob}[\mathbf{s}_i = s_i] &\leq \prod_{i=1}^r \text{Prob}[\mathbf{s}_i \geq s_i] \leq \prod_{i=1}^r (1 - s_i 2^{-b})^{2^t} \\ &\leq \prod_{i=1}^r e^{-s_i 2^{t-b}} = e^{-(\sum s_i) 2^{t-b}} = e^{-T 2^{t-b}} \leq e^{-(S+1) 2^{t-b}} \end{aligned}$$

By choice of the parameters the probability of getting a sum T with $S < T \leq rS$ is therefore limited by $\exp(r \ln(e(2S+1)) - (S+1)2^{t-b})$. Since $\ln(2S+1) \leq S+1$, $r = O(\log k)$ and $2^{t-b} = \omega(\log k)$ this is negligible.

Zero-Knowledge. The zero-knowledge simulator $Z = (Z_0, Z_1)$ in the first stage simply lets H_0 be a (pseudo)random oracle. For the second stage, Z_1 defines H_1 to be consistent with H_0 on previous queries. For any other query to H_1 simulator

⁶ For the simple proof consider first the number of combinations to split T into r strictly positive values. Envision T balls and $T-1$ gaps and place separators in $r-1$ distinct gaps. Assign the i -th value the number of balls between separators $(i-1)$ and i . There are $\binom{T-1}{r-1}$ possibilities to put the separators and therefore the same number of representations. To deal with non-negative integers simply map the integers x to $x+1$ and consider the sum $T+r$ instead of T .

Z_1 , on input x (and YES, the case NO is trivial), first samples 2^t random b -bit strings for each i and assigns them to the t -bit challenges $\text{ch}_i \in \{0,1\}^t$. Let $\tau_i : \{0,1\}^t \rightarrow \{0,1\}^b$ describe this assignment. Let ch_i be the first one (in lexicographic order) obtaining the minimum over all these 2^t values. Z_1 next runs the (wlog.) special zero-knowledge simulator Z_{FS} of the underlying Fiat-Shamir proof r times on x and each ch_i to obtain r tuples $(\text{com}_i, \text{ch}_i, \text{resp}_i)$. It then defines the hash function H_1 for any query $(x, \vec{\text{com}}, i, \text{ch}_i^*, \text{resp}_i^*)$ with $V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i^*, \text{resp}_i^*) = 1$ to be the value $\tau_i(\text{ch}_i^*)$. All other values of H_1 are chosen (pseudo)randomly. The simulator outputs $\pi = (\text{com}_i, \text{ch}_i, \text{resp}_i)_{i=1,2,\dots,r}$ as the proof.

Assume for the moment that the tuples $(\text{com}_i, \text{ch}_i, \text{resp}_i)$ are generated by genuine runs between $P_{\text{FS}}(x, w)$ and $V_{\text{FS}}^{\text{CH}}(x, \text{ch}_i)$, instead of being sampled through $Z_{\text{FS}}(x, \text{ch}_i)$. Recall that $V_{\text{FS}}^{\text{CH}}(x, \text{ch}_i)$ is the special verifier which merely copies ch_i . We only have to show that the simulation of $Z = (Z_0, Z_1)$ on these tuples, called *hybrid simulation*, cannot be distinguished from the genuine behavior between P and V in our protocol. Since the hybrid simulation is efficiently computable given the transcripts, it follows from the special zero-knowledge property of the underlying Fiat-Shamir protocol —and a standard hybrid argument reducing the indistinguishability of r executions to a single one— that our construction inherits the zero-knowledge property.

So let all $(\text{com}_i, \text{ch}_i, \text{resp}_i)$ in the simulator's proof π be generated by running P_{FS} and $V_{\text{FS}}^{\text{CH}}$. Define the following two events in the hybrid simulation. The first event, denoted **InconsistentOracle**, occurs if D_0 at some point queries H_0 about the values $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ for its output x and the i -th proof part in the simulator's output π —which is randomly generated after D 's query. In this case the simulator Z_1 would not have the possibility to program the random oracle H_1 accordingly. The other event **AmbiguousResponse** occurs if D at some point queries H_0 or H_1 about $(x, \vec{\text{com}}, i, \text{ch}, \text{resp})$ and $(x, \vec{\text{com}}, i, \text{ch}, \text{resp}')$ for some i , where $\text{resp} \neq \text{resp}'$ but $V_{1,\text{FS}}(x, \text{com}_i, \text{ch}, \text{resp}) = V_{1,\text{FS}}(x, \text{com}_i, \text{ch}, \text{resp}') = 1$. In this case the simulated hash value of H_1 equals the same value $\tau_i(\text{ch})$ for both values and would be distinguishable from random.

Given that neither **InconsistentOracle** nor **AmbiguousResponse** happens, we claim that the hybrid simulation and executions of our protocol are indistinguishable. This follows easily as the challenge ch_i in each execution is determined in the same way for both P^H and Z_1 . Both algorithms search through the challenges in lexicographic order and the challenges are mapped to random b -bit values (either through H or through τ_i). The remaining parts of the transcript, $\text{com}_i, \text{resp}_i$, are in both cases chosen by the prover $P_{\text{FS}}(x, w)$. The claim now follows under the condition that neither of the two events above occurs because then all other hash values are independently distributed.

It thus remains to bound the probabilities for events **InconsistentOracle** and **AmbiguousResponse**. The former event has negligible probability since the **min-entropy of the commitments is superlogarithmic**. That is, adversary D_0 can query H_0 only about a polynomial number of commitments, and the probability that one of the r random commitments $\text{com}_1, \dots, \text{com}_r$, chosen afterwards,

equals any of those previously selected commitments is negligible. Conditioning on $\neg \text{InconsistentOracle}$, event AmbiguousResponse also has a negligible probability. Otherwise we could easily construct an efficient algorithm, using Z and D as subroutines, to refute the property of unique responses (in the plain model).

Online Extraction. We present a knowledge extractor $K(x, \pi, \mathcal{Q}_H(A))$ that, except with negligible probability over the choice of H , is able to output a witness w to x for an accepted proof $\pi = (\text{com}_i, \text{ch}_i, \text{resp}_i)_{i=1,2,\dots,r}$. Algorithm K browses through the list of queries and answers $\mathcal{Q}_H(A)$ and searches for a query $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ as well another query $(x, \vec{\text{com}}, i, \text{ch}'_i, \text{resp}'_i)$ for a different challenge $\text{ch}_i \neq \text{ch}'_i$ but such that $V_{\text{FS}}(x, \text{com}_i, \text{ch}'_i, \text{resp}'_i) = 1$. If it finds two such queries it runs the knowledge extractor K_{FS} of the Fiat-Shamir proof on these values and copies its output; if there are no such queries then K outputs $\perp$.

It is clear that K succeeds every time it finds two valid queries for the same prefix $(x, \vec{\text{com}}, i)$. Hence, it suffices to bound the probability that there are no two such queries but V still accepts. Consider the set of tuples $(x, \vec{\text{com}})$ such that A queries H about $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ for some $i, \text{ch}_i, \text{resp}_i$ and such that $V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i, \text{resp}_i) = 1$. Note that we can neglect tuples with invalid proofs $(\text{com}_i, \text{ch}_i, \text{resp}_i)$. Letting $Q = |\mathcal{Q}_H(A)|$ and counting the tuple in A 's final output (x, π) as well, there are at most $Q + 1$ many of these tuples $(x, \vec{\text{com}})$.

Fix one of the tuples for the moment, say, $(x, \vec{\text{com}})$. By assumption, for this tuple and any i algorithm A never queries H about two values $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$, $(x, \vec{\text{com}}, i, \text{ch}'_i, \text{resp}'_i)$ with $\text{ch}_i \neq \text{ch}'_i$ which V_{FS} would accept. Similarly, we can assume that A never queries about $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$, $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}'_i)$ with $\text{resp}_i \neq \text{resp}'_i$, else this would contradict the property of unique responses and can therefore happen with negligible probability only. This allows us to assign a set of unique values $s_1, \dots, s_r$ to $(x, \vec{\text{com}})$ such that s_i equals $H(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ if A queries about some tuple $(x, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$, and a random b -bit value if A never queries about any such tuple. Conclusively, the values $s_1, \dots, s_r$ assigned to $(x, \vec{\text{com}})$ are all random and independent.

Given such an assignment we calculate the probability that the sum does not exceed the threshold value S . We consider again the $\binom{T+r-1}{r-1}$ combinations to represent a sum $T \leq S$ with r values $s_1, \dots, s_r \geq 0$. There are

$$\sum_{T=0}^S \binom{T+r-1}{r-1} \leq (S+1) \cdot \binom{S+r-1}{r-1} \leq (S+1) \cdot \left(\frac{e(S+r)}{r-1} \right)^{r-1}$$

possibilities. Since $S = O(r)$ we have $S + r \leq c(r-1)$ for some constant c , and the number of combinations is bounded above by $(S+1) \cdot 2^{\log(ec)r}$, which is polynomial in k . Since $s_1, \dots, s_r$ are random, the probability of obtaining such a sum $T \leq S$ is this number of combinations divided by 2^{br} . By the choice of parameters this is negligible.

Finally, we extend the analysis from a fixed query to the set of the $Q + 1$ possibilities $(x, \vec{\text{com}})$. Since Q is polynomial the probability of finding a valid proof among this set for which the extractor also fails is bounded by $(Q+1)(S+1) \cdot 2^{(\log(ec)-b)r}$. This remains negligible. $\square$

We remark that the upper bounds derived on the number of representations of T with r integers, for completeness and extraction, have not been optimized. Moreover, we providently note that our knowledge extractor only needs the hash queries in $\mathcal{Q}_H(A)$ with prefix $(x, \vec{\text{com}})$ to extract the witness for theorem x ; all other queries are irrelevant to K .

Comparison to Hash-Tree Construction. We compare our construction with on-line extractors based on hash trees. Recall that, for the hash tree construction, in each of the r repetitions the prover computes the commitment com and all possible responses $\text{resp}(\text{ch})$ for challenges $\text{ch} \in \{0, 1\}^b$. Hash values of all 2^b responses are placed as leaves in a hash tree, and a binary tree of depth b is computed. This requires altogether $2^b + 2^b - 1 \approx 2^{b+1}$ hash function evaluations. The challenge is computed as the hash value over all commitments and tree roots, and in each tree the corresponding leaf is opened together with the siblings on the path.

To compare the efficiency of the two approaches, we set $b = 9$, $t = 12$, $r = 10$ and $S = 10$ for our construction and $b' = 8$ and $r' = 10$ for the hash-tree construction. Then the total number of hash function evaluations is roughly $r \cdot 2^9$ in both cases, and the number of executions of the underlying protocol are identical. In favor of the hash tree construction it must be said that our solution requires twice as many response computations on the average, though.

We have already remarked that the communication complexity of the hash-tree construction is significantly larger than for our construction, i.e., the partly disclosed hash trees add $br = 90$ hash values (typically of 160 or more bits) to the proof, while our solution does not add any communication overhead. As for the extraction error, the exact analysis for our construction with the given parameters shows that the extractor fails with probability at most $Q \cdot 2^{-72}$ where Q is the maximal number of hash queries (assuming that finding distinct responses is beyond feasibility). The extraction for the hash-tree construction basically fails only if one manages to guess all r challenges in advance and to put only one correct answer in each tree. This happens with probability approximately $Q/2^{br} = Q \cdot 2^{-80}$ and is only slightly smaller than for our construction. Yet, extraction in the hash-tree construction also requires that no collisions for the hash function are found. Finally, we note that the honest prover always manages to convince the honest verifier for the hash-tree construction whereas our protocol has a small completeness error.

Properties. Concerning the type of zero-knowledge, if there is a unique response for each x, com, ch , then our transformation converts an honest-verifier perfect zero-knowledge protocol into a statistical zero-knowledge one (against malicious verifiers). The small error is due to the negligible collision probability of commitments and applies to the standard Fiat-Shamir transformation as well.

As for proving logical combinations, given two interactive Fiat-Shamir protocols for two relations W^0, W^1 it is known [CP92,CDS95,DDPY94] how to construct three-move proofs showing that one knows at least one of the witnesses to x^0, x^1 (i.e., prove OR), or one can also show that one knows both witnesses (i.e., prove AND). Since the derived protocols in both cases preserve

the zero-knowledge and extraction property, and therefore constitute themselves Fiat-Shamir proofs of knowledge, our conversion can also be carried out for proving such logical statements.

3.2 Simulation-Soundness

Before specifying our signature scheme we first prove formally that our non-interactive proof is simulation sound. That is, even if the zero-knowledge simulator has simulated several proofs for adversarial chosen theorems, the online extractor can still extract the witness from the adversarial proof for a valid theorem (as long as either the theorem or the proof is new). For this we also extend the notion of zero-knowledge to the case where the distinguisher sees multiple zero-knowledge proofs. We remark that we take advantage of special properties of our protocol to achieve both properties, i.e., they are not known to follow immediately from the basic properties.

Theorem 3. *Let $(P_{\text{FS}}, V_{\text{FS}})$ be an interactive Fiat-Shamir proof of knowledge for relation W and let (P, V) be derived through Construction 1. Then the protocol has the following additional properties:*

[Multiple Zero Knowledge.] *There exists a probabilistic polynomial-time algorithm Z , the zero-knowledge simulator, such that for any probabilistic polynomial-time algorithm D the following distributions are computationally indistinguishable:*

- *Let H be a random oracle. Set $\pi_0 = \epsilon$ and $\delta_0 = k$. For $i = 1, 2, \dots, n$ (until D stops) repeat $(x_i, w_i, \delta_i) \leftarrow D^H(i, \pi_{i-1}, \delta_{i-1})$ where $\pi_i \leftarrow P^H(x_i, w_i)$ if $(x_i, w_i) \in W_k$ and $\perp$ otherwise. Copy D 's final output.*
- *Let $(H_0, \sigma_0) \leftarrow Z(0, k)$, $\pi_0 = \epsilon$ and $\delta_0 = k$. For $i = 1, 2, \dots, n$ (until D stops) repeat $(x_i, w_i, \delta_i) \leftarrow D^{H_{i-1}}(i, \pi_{i-1}, \delta_{i-1})$ where $(H_i, \sigma_i, \pi_i) \leftarrow Z(i, x_i, \sigma_{i-1}, \text{YES})$ if $(x_i, w_i) \in W_k$ and $(H_i, \sigma_i, \pi_i) \leftarrow Z(i, x_i, \sigma_{i-1}, \text{NO})$ otherwise. Copy D 's final output.*

[Simulation Soundness.] *There exists a probabilistic polynomial-time algorithm K , the online extractor, such that the following holds for every probabilistic polynomial-time algorithm B . Let $(H_0, \sigma_0) \leftarrow Z(0, k)$, $\pi_0 = \epsilon$ and $\beta_0 = k$. For $i = 1, 2, 3, \dots, n$ (until B stops) repeat $(x_i, w_i, \beta_i) \leftarrow B^{H_{i-1}}(i, \pi_{i-1}, \beta_{i-1})$ where $(H_i, \sigma_i, \pi_i) \leftarrow Z(i, x_i, \sigma_{i-1}, \text{YES})$ if $(x_i, w_i) \in W_k$ and $(H_i, \sigma_i, \pi_i) \leftarrow Z(i, x_i, \sigma_{i-1}, \text{NO})$ otherwise. Let (x, π) be B 's final output and $\mathcal{Q}_B(H)$ be the communication of B with oracles $H_0, H_1, \dots, H_n$. Let $w \leftarrow K(x, \pi, \mathcal{Q}_B(H))$. Then, as a function of k ,*

$$\text{Prob}[(x, w) \notin W_k \wedge V^{H_n}(x, \pi) = 1 \wedge (x, \pi) \notin \{(x_1, \pi_1), \dots, (x_n, \pi_n)\}] \gtrsim 0.$$

Proof. The multiple zero-knowledge property follows easily from the basic simulator for single theorems, because for each query the more sophisticated simulator here computes independent commitments—which are unique with overwhelming probability—and can therefore adapt the random oracle values accordingly.

We show simulation soundness. We claim that the same extractor K as in the basic case works. So assume towards contradiction that there exists an algorithm B refuting the property for this extractor K . From B we construct an algorithm A contradicting the basic extraction property of system (P, V) , i.e., A manages to generate a valid proof for a theorem without having seen other proofs before, but the knowledge extractor cannot extract a witness with sufficiently close probability.

We first show that we can exclude some trivial attacks. Namely, the probability that B only changes one of the responses in the proof $\pi_j = (\text{com}_{ij}, \text{ch}_{ij}, \text{resp}_{ij})_{i=1,2,\dots,r}$ of a previous theorem $x = x_j$, and still succeeds, is negligible. Else it would be straightforward to construct an algorithm refuting the property of unique responses for $(P_{\text{FS}}, V_{\text{FS}})$, where this algorithm works in the plain model and uses the zero-knowledge simulator to emulate the random oracle and to generate proofs. Similarly, if B copies a theorem $x = x_j$ but changes one of the (simulated) challenge-response pairs $(\text{ch}_{ij}, \text{resp}_{ij})$ in the proof π_j to another valid transcript $(\text{ch}'_{ij}, \text{resp}'_{ij})$, then the extractor can already derive the witness from these two valid transcripts. In the sequel we therefore condition on B succeeding by producing a new tuple (x, com) , distinct from all previous theorem-commitment pairs (x_j, com_j) .

Algorithm $A^H(k)$, contradicting the basic extraction property of the underlying protocol, simulates B as follows. A sets $\pi_0 = \epsilon$ and $\beta_0 = k$. For $i = 1, 2, \dots, n$ it runs B on input $(i, \pi_{i-1}, \beta_{i-1})$. In each loop the random oracle queries of B are relayed between A 's oracle H and B . If B outputs (x_i, w_i, β_i) at the end of one round, then A checks (in polynomial time) if $(x_i, w_i) \in W_k$; if so, it runs the prover P (with oracle access to H) to compute $\pi_i \leftarrow P^H(x_i, w_i)$; if not, it sets $\pi_i \leftarrow \perp$. When B finally outputs (x, π) algorithm A outputs (x, π) , too.

We construct a distinguisher D against the multiple zero-knowledge property to show that the extraction error in A 's experiment is negligibly close to the one in B 's experiment. Since the latter is assumed to be noticeable, so is the former, contradicting the extraction error of the underlying system (P, V) .

Algorithm D proceeds in rounds, where at the end of each round D outputs (x_i, w_i, δ_i) and expects a proof at the beginning of the next round. D either experiences simulated proofs from the zero-knowledge simulator (and a simulated random oracle), or gets genuine proofs from the original prover (and has access to a truly random oracle). D uses B , which also proceeds in rounds, as a subroutine. Each time B outputs (x_i, w_i, β_i) at the end of one round, D copies this output and finishes this round as well. During each round every hash oracle query of B is forwarded to D 's (simulated or real) hash oracle, and the answer is returned to B ; all queries and answers are recorded in δ_i . When B eventually returns (x, π) and stops, D invokes V^H on (x, π) , verifies (in polynomial time) if $(x, w) \notin W_k$ for $w \leftarrow K(x, \pi, \mathcal{Q}_H(B))$ (where $\mathcal{Q}_H(B)$ is the recorded list of queries and answers of B and the hash oracle), and D checks that $(x, \pi) \notin \{(x_j, \pi_j)\}$. Algorithm D outputs 1 if and only if all tests succeed.

For the analysis, if D communicates with the zero-knowledge simulator then it outputs 1 with the extraction error in B 's experiment. If, on the other hand, D

interacts with the original prover and a random oracle, then we claim that D returns 1 with the extraction error in A 's experiment. The only difference between A 's experiment and the D 's simulation in this case lies in the hash queries of the algorithms: A 's list $\mathcal{Q}_H(A)$ also includes queries A uses to internally simulate the prover P^H , but those queries do not appear in D 's records. As noted before, our basic extractor K only needs to see A 's hash queries corresponding to the tuples $(x, \vec{\text{com}})$. All queries referring to $(x_1, \vec{\text{com}}_1), \dots, (x_n, \vec{\text{com}}_n)$ different from this tuple (which include all queries of P^H) can be neglected without decreasing the extraction probability. In conclusion, if the probabilities for D 's two experiments would be far apart then D would contradict the multiple zero-knowledge property. Hence, they must be negligibly close and the claim follows. $\square$

3.3 Constructing Signature Schemes

In order to derive a signature scheme from the three-move proof of knowledge the classical Fiat-Shamir transformation suggests to compute the challenge as the hash value over com and the message to be signed. In our case, the challenge is not derived as a hash value. We can, however, incorporate m by computing the b -bit hash values over $(x, m, \vec{\text{com}}, i, \text{ch}_i, \text{resp}_i)$ for the public key x , i.e., m becomes part of the theorem. Indeed, if we have an interactive proof of knowledge for relation W then this also constitutes a proof of knowledge for the relation $W_k^{\text{msg}} = \{((x, m), w) \mid (x, w) \in W_k\}$. The protocol also inherits completeness and the multiple zero-knowledge property and our transformation also yields a simulation-sound non-interactive proof system.

For our signature scheme we assume that the underlying relation W of the proof of knowledge is accompanied by an efficiently samplable, yet hard to invert procedure generating (x, w) . For example, for the discrete-logarithm problem and parameter $k = (\mathcal{G}, q, g)$ this procedure picks $w \leftarrow \mathbb{Z}_q$ and computes $x \leftarrow g^w$. More formally, we say that the relation W has a *one-way instance generator* $\mathcal{I}$ if for any parameter k algorithm $\mathcal{I}$ returns in probabilistic polynomial-time $(x, w) \in W_k$, but such that for any probabilistic polynomial-time algorithm I , for $(x, w) \leftarrow \mathcal{I}(k)$ and $w' \leftarrow I(x)$ the probability $\text{Prob}[(x, w') \in W_k]$ is negligible (as a function of k).

Construction 4. Let $(P_{\text{FS}}, V_{\text{FS}})$ be an interactive Fiat-Shamir proof of knowledge for relation W with one-way instance generator $\mathcal{I}$, and let (P, V) be derived through Construction 1. Define the following signature scheme $(\text{KGen}, \text{Sig}, \text{Ver})$ in the random oracle model:

- [Key Generation.] Algorithm $\text{KGen}(k)$ generates a key pair $(pk, sk) \leftarrow \mathcal{I}(k)$.
- [Signing.] Algorithm Sig^H for inputs pk, sk and message $m \in \{0, 1\}^*$ computes a proof of knowledge $s \leftarrow P^H((pk, m), sk)$ for the relation W_k^{msg} and returns s as the signature.
- [Verification.] Algorithm Ver^H for input pk, m and s returns the verifier's decision $V^H((pk, m), s)$ for relation W_k^{msg} .

Proposition 1. *The signature scheme $(\text{KGen}, \text{Sig}, \text{Ver})$ in Construction 4 is existentially unforgeable against adaptive chosen-message attacks (in the random oracle model). In particular, if there is an algorithm mounting a chosen-message attack in time T with success probability ϵ , then there is an algorithm inverting $\mathcal{I}$ in time $T' = O(T)$ with probability $\epsilon' \approx \epsilon$.*

In fact, our result even shows that the derived signature scheme is strongly unforgeable, i.e., it is also infeasible to take a message-signature pair (m, s) and produce a new signature for this message m .

Proof. Security follows from the simulation soundness of the proof of knowledge. Suppose that there is a successful attacker A on the signature scheme which forges signatures with noticeable probability. We show that this would contradict the one-wayness of the instance generator.

First note that if A faces the simulator of the multiple zero-knowledge property instead of the honest prover and a genuine random oracle, then A 's success probability of forging signatures cannot drop significantly. Else it would be easy to derive a distinguisher for the zero-knowledge property. Thus, the adversary has also noticeable success probability in this *hybrid experiment* with simulated proofs and oracles.

Next, we explain how to use A to contradict the one-wayness of $\mathcal{I}$. Given the public part pk , generated as $(pk, sk) \leftarrow \mathcal{I}(k)$, initialize the zero-knowledge simulator to obtain oracle H_0 . Run the attacker on pk and with oracle access to H_0 . Each time the attacker asks for a signature for a message m_i we run the zero-knowledge simulator on (pk, m_i, YES) to obtain a proof s_i (i.e., a signature) for this theorem and to get oracle H_i . We answer with s_i and continue the simulation with oracle H_i . If the adversary finally outputs a forgery (m, s) then we run the knowledge extractor K on $(pk, m), s$ and the list of oracle queries A has made. Return the extractor's output sk' .

It is easy to see that if A manages to output a valid signature s (i.e., an accepted proof) for a new message m (i.e., a new theorem (pk, m)) with noticeable probability in the hybrid experiment, then the knowledge extractor can efficiently extract a matching secret key sk' (i.e., a witness) with noticeable probability. Specifically, the probability of A succeeding while the extractor does not return a witness is negligible by simulation soundness. We derive the contradiction to the one-wayness of $\mathcal{I}$ as we can efficiently return a witness with noticeable probability. $\square$

The result of the theorem holds more generally if the extractor merely computes a function $f(w)$ of the witness, as long as computing such a value $f(w)$ is infeasible.

3.4 Lower Bound for Hash Queries of Online Extractors

In this section we show our superlogarithmic lower bound on the number of hash function evaluations for non-programming online extractors. For notational convenience we let $f(k) = O_K(\log k)$ or $f(k) = \text{poly}_K(k)$ refer to a function $f(k)$

which grows only logarithmically or polynomially, restricted to all $k \in K$, i.e., there is a constant c such that $f(k) \leq c \log k$ or $f(k) \leq k^c$ for all $k \in K$. For any $k \notin K$ the function f might exceed these bounds.

Recall from the previous section that a one-way instance generator $\mathcal{I}$ for a relation W generates random x such that finding a witness w to x is infeasible.

Proposition 2. *Let (P, V) be a non-interactive zero-knowledge proof of knowledge for relation W with an online extractor K in the random oracle model. Let $\rho = \rho(k)$ and $\nu = \nu(k)$ be the maximum number of hash oracle queries the prover P resp. the verifier V makes to generate and to check a proof π . Then $\max_{v=0,1,\dots,\nu} \binom{\rho}{v} = \text{poly}_K(k)$ for an infinite set K implies that W does not have a one-way instance generator $\mathcal{I}$.*

Clearly, $\binom{\rho}{v}$ obtains its maximum at $\binom{\rho}{\lceil \rho/2 \rceil}$, where $\lceil \rho/2 \rceil$ is the rounded-off integer of $\rho/2$, and if $\rho = O_K(\log k)$ for an infinite set K , then $\binom{\rho}{\lceil \rho/2 \rceil} \leq (2e)^{\lceil \rho/2 \rceil} = \text{poly}_K(k)$ for the same set K , and the requirements of the proposition are satisfied. This implies that $\rho = \omega(\log k)$ must grow superlogarithmically for a one-way instance generator. Similarly, if the verifier only makes a constant number of hash function queries then the prover must perform a superpolynomial number of hash function evaluations, or else the instance generator cannot be one-way.

Proof. The high level idea of the proof is that, under the assumption that $\max_v \binom{\rho}{v} = \text{poly}_K(k)$ is polynomial, replacing the hash queries $\mathcal{Q}_H(P)$ the prover makes to generate the proof π by the queries $\mathcal{Q}_H(V)$ the verifier makes to verify the proof suffices to extract the witness. Specifically, we prove that $K(x, \pi, \mathcal{Q}_H(V))$ then returns a witness w with noticeable probability. Replacing the original proof by an indistinguishable one from the zero-knowledge simulator $Z(x)$ (without access to w) and running $K(x, \pi, \mathcal{Q}_H(V))$ on this proof implies that we can compute the witness w with noticeable probability from x alone.

We now turn the above idea into a formal proof. Suppose that there is an instance generator $\mathcal{I}$ for relation W . We construct an algorithm I that succeeds in finding a witness for a random instance x with noticeable probability, and therefore the generator cannot be one way. Inverter I works as follows:

- Algorithm I gets as input x where $(x, w) \leftarrow \mathcal{I}(k)$.
- I invokes the zero-knowledge simulator to generate $(H, \pi) \leftarrow Z(x)$.
- The inverter next runs the verifier $V^H(x, \pi)$ and records all communication of V with H in $\mathcal{Q}_H(V)$.
- I executes the extractor $K(x, \pi, \mathcal{Q}_H(V))$ and outputs the answer w .

For the analysis we first construct, in a thought experiment, a prover P' from P which tries to make at most those queries V makes for verification. P' , on input w, x and with access to H , first picks a random subset Q of all subsets of $\{1, 2, \dots, \rho\}$ of size at most ν , e.g., by choosing a random v between 0 and ν and then picking a subset of size v . It next emulates P on w, x . If P generates the i -th query to the oracle then, for $i \in Q$, prover P' forwards this query to H

and returns the answer to P ; if $i \notin Q$ then P' simply returns a random value from H 's range.⁷ If P at the end outputs a proof π then P' runs the (wlog.⁸) deterministic verifier V^H by relaying communication between V and the genuine oracle H and eventually outputs π (even if V aborts or rejects).

For notational convenience denote by H' the function which is defined by H and P' actions, i.e., which agrees on H on all queries except for the ones for which P' gave random answers instead. Clearly, P' may sometimes fail to provide a proof that V^H accepts, especially if V queries H about a value for which H' is different. But since $\max_v \binom{p}{v} = \text{poly}_K(k)$ for an infinite set K , prover P' predicts the subset of queries V makes among the queries of P correctly (event **Consistent-Oracle**) with probability $1/p(k)$ for all $k \in K$ and some polynomial $p(k)$. Given that the guess is right, V 's view is identical when run with H or with H' . Hence, taking the negligible completeness error of the original protocol into account, V^H accepts the proof of P' with probability at least $1/2p(k)$ for all sufficiently large $k \in K$.

It follows that the knowledge extractor K , run on the queries of P' to H , returns a valid witness with probability negligibly close to $1/2p(k)$, which is certainly larger than $1/4p(k)$ for all large $k \in K$. Furthermore, given **Consistent-Oracle** we have $\mathcal{Q}_H(P') = \mathcal{Q}_H(V)$; the inclusion $\mathcal{Q}_H(P') \subseteq \mathcal{Q}_H(V)$ follows from the correct guess and equality from the fact that P' conclusively runs the deterministic V on π . Under this condition P' also generates the same distribution on π as P does (given $x, \mathcal{Q}_H(V)$) and the extractor's view is identical in that case. Hence, $K(x, \pi, \mathcal{Q}_H(V))$ also returns a witness with noticeable probability $1/4p(k)$ in the actual protocol between P and V .

The final step is to let the zero-knowledge simulator $Z(x)$ generate H_Z, π_Z instead of P . For x the probability of $K(x, \pi_Z, \mathcal{Q}_{H_Z}(V))$ extracting a witness can only be negligibly smaller than for $K(x, \pi, \mathcal{Q}_H(V))$. Otherwise it would be straightforward to construct a distinguisher for the zero-knowledge property. It follows that our inverter I finds a witness to a given x with noticeable probability at least $1/8p(k)$ for large $k \in K$ and the instance generator cannot be one-way. $\square$

Optimality of the Bound. Our lower bounds make essential use of the fact that the extractor cannot program the random oracle. In fact, if K was allowed to choose oracle values, then the oracle H (with unrestricted output length) could be defined to generate a sufficiently large common reference string and to run a non-interactive zero-knowledge proof of knowledge with online extractor in the standard model [DP92]. A single hash function evaluation would then suffice.

Also, the superlogarithmic bound cannot be improved for non-programming extractors. Namely, if we run the hash-tree construction or an easy modifica-

⁷ As usual, P' simulates oracle queries consistently by keeping state of previous queries and providing identical answers to identical queries.

⁸ Every probabilistic verifier can be replaced by a deterministic one. This only increases the verifier's number of hash function queries by one and does not change our asymptotical result.

tion of our solution for binary challenges and superlogarithmic r , then we get a negligible extraction error and make only $O(r)$ hash function queries.

4 Application to Group Signatures

In this section we show how to lift the CPA-anonymous group signature scheme by Boneh et al. [BBS04] to a fully anonymous one. As explained in the introduction, the idea is to append a non-interactive proof of knowledge with online extractor for an ElGamal-like encryption. Although we give a brief introduction to group signatures we refer the reader to the work by Bellare et al. [BMW03] for a comprehensive overview about (the security of) group signatures. Recall from the introduction that the two important security properties are full anonymity, the impossibility of identifying the author of a signature, and full traceability, the impossibility of generating a signature which cannot be traced to the right origin.

Very roughly, a group signature scheme consists of a (fixed) set of users, each user receiving a secret through an initial *key generation* phase carried out by a trusted third party. In addition, a public group key is established in this phase. Each user can run the *sign protocol* to generate a signature on behalf of the group. This signature is *verifiable* through the group's public key, yet outsiders remain oblivious about the actual signer. Only the group manager can revoke this anonymity and *open* the signature through an additional secret key.

The original scheme by Boneh works over bilinear group pairs (G_1, G_2) where deciding the Diffie-Hellman problem is easy. That is, for groups G_1, G_2 of prime order q generated by g_1, g_2 there is an efficiently computable isomorphism $\psi : G_2 \rightarrow G_1$ with $\psi(g_2) = g_1$, and an efficiently computable non-degenerated bilinear mapping e with $e(u^a, v^b) = e(u, v)^{ab}$ for all $u \in G_1, v \in G_2$ and $a, b \in \mathbb{Z}_q$.

For the security of the scheme it is assumed that the q -strong Diffie-Hellman problem —given $(g_1, g_2, g_2^\gamma, g_2^{\gamma^2}, \dots, g_2^{\gamma^q})$ find $(g_1^{\gamma^{+x}}, x)$ for *any* $x \in \mathbb{Z}_q^*$ — is intractable. See [BB04] for more details. It is also presumed that the decision linear assumption in G_1 holds, namely that it is infeasible to distinguish tuples $(u, v, h, u^a, v^b, h^{a+b})$ and (u, v, h, u^a, v^b, h^c) for $u, v, h \leftarrow G_1$ and $a, b, c \in \mathbb{Z}_q$. This assumption implies that ElGamal-like encryptions $(u^a, v^b, h^{a+b} \cdot m)$ of messages m under public key (u, v, h) are semantically secure.

In the original scheme of Boneh et al. [BBS04] the group's public key contains a value $w = g_2^\gamma$ and each user receives a pair (A_i, x_i) with $A_i = g_1^{1/(\gamma+x_i)}$ as the secret key. In addition, the group manager's public key consists of a public encryption key (u, v, h) such that $u^{\xi_1} = v^{\xi_2} = h$ for secret key ξ_1, ξ_2 . To sign a message m the user encrypts A_i with the manager's public key as $T_1 \leftarrow u^a, T_2 \leftarrow v^b$ and $T_3 \leftarrow A_i h^{a+b}$ for random $a, b \leftarrow \mathbb{Z}_q$. In addition, the signer also computes a non-interactive proof τ (in the random oracle model) that (T_1, T_2, T_3) encrypts such an A_i with $e(A_i, w g_2^{x_i}) = e(g_1, g_2)$. The details of this zero-knowledge proof are irrelevant for our discussion here, we merely remark that the message m enters in this proof and that an independent random oracle is needed for this part. To verify a signature one verifies this proof τ . To

revoke anonymity the group manager verifies the signatures and then decrypts $A_i = T_3/T_1^{\xi_1}T_2^{\xi_2}$ and recovers the user's identity through A_i .

We now augment the original scheme by our proof of knowledge for the ElGamal encryption:

Construction 5. Define the following group signature scheme:

- [Key Generation.] Compute the public key as before by picking a bilinear group pair $\mathcal{G} = (G_1, G_2)$ and generators g_1, g_2, h . Sample $\xi_1, \xi_2, \gamma \leftarrow \mathbb{Z}_q^*$ and let $u^{\xi_1} = v^{\xi_2} = h$ and $w = g_2^\gamma$. The public key gpk consists of $(\mathcal{G}, g_1, g_2, h, u, v, w)$. Each of the n users obtains some $x_i \leftarrow \mathbb{Z}_q^*$ and $A_i = g_1^{1/(\gamma+x_i)}$ as the secret key. The group manager receives $(\xi_1, \xi_2, A_1, \dots, A_n)$ as the secret key.
- [Signing.] To sign a message $m \in \{0, 1\}^*$ under a secret key (A_i, x_i) the user takes the group key $\text{gpk} = (\mathcal{G}, g_1, g_2, h, u, v, w)$ and does the following:
- As in the original scheme pick $a, b \leftarrow \mathbb{Z}_q$ and encrypt A_i under the group manager's public key, $T_1 \leftarrow u^a$, $T_2 \leftarrow v^b$ and $T_3 \leftarrow A_i h^{a+b}$.
 - Compute as before a non-interactive proof τ that $A_i = g_1^{1/(\gamma+x_i)}$ is encrypted in (T_1, T_2, T_3) for some $x_i \in \mathbb{Z}_q$, involving the message m .
 - Additionally, compute a non-interactive zero-knowledge proof of knowledge π for α, β , i.e., run P^H on $(\text{gpk}, T_1, T_2, T_3, \tau, m, \alpha, \beta)$ for relation $W_k = \{((\text{gpk}, T_1, T_2, T_3, \tau, m), (\alpha, \beta)) \mid u^\alpha = T_1, v^\beta = T_2\}$ to obtain π .
 - Output $(T_1, T_2, T_3, \pi, \tau)$ as the signature to m .
- [Verification.] To verify a signature $(T_1, T_2, T_3, \pi, \tau)$ for a message m run the original verifier of the signature scheme and also run the verifier V^H of the non-interactive proof of knowledge on $(\text{gpk}, (T_1, T_2, T_3, \tau, m), \pi)$. Accept if both verifications succeed.
- [Open.] To reveal the identity of a signer for a signature $(T_1, T_2, T_3, \pi, \tau)$ the group manager first verifies the validity of the signature (including the proof π). If correct, then the manager decrypts as in the original scheme to recover some $A = T_3/(T_1^{\xi_1}T_2^{\xi_2})$ and compares this value to the list of A_i 's to find the user index i .

For system parameters suggested in [BBS04], namely, $|q| = 170$ bits and $|G_1| = 171$ bits, the original signature length is 1,533 bits. If we use the same values $b = 9, r = S = 10, t = 12$ as in the previous section for our proof system, then our scheme adds about $2r \cdot 170 + rt = 3,520$ bits to signatures through the r repetitions of the atomic protocol for proving the AND of the two discrete logarithms. This proof requires $2r$ answers in $\mathbb{Z}_q$ (as usual in the discrete logarithm case, the commitments are not included in the proof π) and r challenges of t bits. Although the communication complexity of this new scheme is significantly larger, it is still superior to RSA-based group signatures where signatures easily exceed 10,000 bits [ACJT00].

Interestingly, we still expect our version of the group signature scheme to be more efficient than the RSA-based scheme in [ACJT00], where half a dozen exponentiations with large exponents of more than thousand bits have to be carried out without Chinese Remainder. According to implementation results in [DMPW98] a single exponentiation for elliptic curves is estimated to be about

ten times faster than such RSA exponentiations; the exact figures of course depend on implementation details.

Proposition 3. *Under the Decision Linear Diffie-Hellman and the q -strong Diffie-Hellman assumption the group signature scheme in Construction 5 is a fully anonymous and fully traceable group signature scheme in the random oracle model.*

Proof. A careful analysis reveals that the proof of the traceability property basically carries over from [BBS04]. We omit this part because of the similarity. It thus suffices to show that the new scheme achieves full anonymity. The proof is an adaptation of the original proof in [BBS04], i.e., if there was an algorithm A refuting the anonymity property we can derive an algorithm B breaking the semantic security of the linear encryption scheme.

More precisely, suppose A breaks the anonymity of the group signature scheme. That is, A first asks the group manager about some signatures to reveal the identity, then outputs a message m and two indices i_0, i_1 , from which one is picked at random according to a bit b , and then user i_b creates a challenge signature s for m . The adversary may now continue to query the open oracle (for signatures different from s) and finally outputs a guess b' for b . By assumption, the probability of $b = b'$ is noticeably larger than $1/2$.

We describe our algorithm B attacking the semantic security of the encryption scheme. Algorithm B is given a tuple $(\mathcal{G}, u, v, h)$ and generates the complementary data for the group signature schemes (i.e., γ , w and all pairs (A_i, x_i)) and starts a simulation for A on these data. B also initializes the zero-knowledge simulator of our proof system (and possibly the simulator for generating τ) to get a simulated random oracle H (and an independent one for the τ -part). During the simulation B relays and records the communication between A and the random oracles, especially for H . If A makes a query about message m and signature $(T_1, T_2, T_3, \pi, \tau)$ to the group manager, then B verifies the correctness of the signature; if correct, B runs the knowledge extractor K of the proof system on $gpk, (T_1, T_2, T_3, \tau, m)$ and the list $\mathcal{Q}_H(A)$ of previously recorded hash queries. If K returns a valid witness $w = (\alpha, \beta)$ then B computes $A = T_3/h^{\alpha+\beta}$ and computes the index as the group manager would; in any other case return $\perp$.

If A outputs a challenge query (m, i_0, i_1) then B forwards (A_{i_0}, A_{i_1}) to its challenge oracle to receive a ciphertext (T_1, T_2, T_3) of one of the values, the choice made according to a secret random bit d . B next runs the (statistical) zero-knowledge simulators to generate proofs π and τ . It returns $(T_1, T_2, T_3, \pi, \tau)$ as the signature and continues A 's simulation. All subsequent hash and open queries are treated as in the first phase. If A eventually outputs a guess b' for b , then B copies this output as a prediction d' for d .

For the analysis note that B 's running time is essentially the same as for A . Algorithms B only needs to perform as many additional runs of the extractor K as A puts open queries, plus B needs some extra time to run the zero-knowledge simulators and to maintain the hash query list. To determine B 's success probability we take a look at the first open query of A . The simulation only yields a

different result than in an actual attack, if the signature is valid (in particular, π is correct) and some A_i is encrypted, yet K fails to output the witness for the values T_1, T_2 . The probability of this is negligible by the extraction property, though. Conditioning on the validity of the simulation for this query we can set the argument forth to the following queries. Because of the simulation soundness this argument also holds for open request A makes after having received the challenge signature, as either the theorem or the proof π has to be new. In summary, the simulation fails with negligible probability only.

It follows that B predicts d correctly if A does, except with some negligible probability. This, however, contradicts the semantic security of the underlying encryption scheme. $\square$

5 Open Problems

Currently, non-interactive proofs with online extractors, including the one in this paper, are less efficient than “regular” ones. And although we have shown that for some type of online extractors this loss in efficiency is inevitable, and even if our solution is optimal in some cases, our lower bound does not completely rule out that there are better constructions. Two possible venues for improvements are to take advantage of extractors programming the random oracle, or to accept a large number of hash function evaluations but keeping the number of repetitions of the underlying protocol small. Instead of looking into general transformations, improvements for some popular cases such as RSA or discrete logarithms, possibly exploiting special properties of the underlying protocol, are also of interest.

Acknowledgments

We thank the anonymous reviewers of Crypto 2005 for very comprehensive comments and suggestions.

References

- [Abe04] Masayuki Abe. *Combining Encryption and Proof of Knowledge in the Random Oracle Model*. *The Computer Journal*, 47(1):58–70, 2004.
- [ACJT00] Giuseppe Ateniese, Jan Camenisch, Marc Joye, and Gene Tsudik. *A Practical and Provably Secure Coalition-Resistant Group Signature Scheme*. *Advances in Cryptology — Crypto 2000*, Volume 1880 of *Lecture Notes in Computer Science*. Springer-Verlag, 2000.
- [BB04] Dan Boneh and Xavier Boyen. *Short Signatures Without Random Oracles*. *Advances in Cryptology — Eurocrypt 2004*, Volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Springer-Verlag, 2004.
- [BBP04] Mihir Bellare, Alexandra Boldyreva, and Adriana Palacio. *An Un-Instantiable Random-Oracle-Model Scheme for a Hybrid-Encryption Problem*. *Advances in Cryptology — Eurocrypt 2004*, Volume 3027 of *Lecture Notes in Computer Science*. Springer-Verlag, 2004.

- [BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. *Short Group Signatures*. Advances in Cryptology — Crypto 2004, Volume 3152 of Lecture Notes in Computer Science, pages 41–55. Springer-Verlag, 2004.
- [BGR98] Mihir Bellare, Juan Garay, and Tal Rabin. *Fast Batch Verification for Modular Exponentiation and Digital Signatures*. Advances in Cryptology — Eurocrypt’98, Volume 1403 of Lecture Notes in Computer Science, pages 236–250. Springer-Verlag, 1998.
- [BMW03] Mihir Bellare, Daniele Micciancio, and Bogdan Warinschi. *Foundations of Group Signatures: Formal Definitions, Simplified Requirements, and a Construction Based on General Assumptions*. Advances in Cryptology — Eurocrypt 2003, Volume 2656 of Lecture Notes in Computer Science, pages 614–629. Springer-Verlag, 2003.
- [BR93] M. Bellare and P. Rogaway. *Random Oracles are Practical: A Paradigm for Designing Efficient Protocols*. Proceedings of the Annual Conference on Computer and Communications Security (CCS). ACM Press, 1993.
- [CDS95] Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. *Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols*. Advances in Cryptology — Crypto’94, Volume 839 of Lecture Notes in Computer Science, pages 174–187. Springer-Verlag, 1995.
- [CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. *The Random Oracle Methodology, Revisited*. Proceedings of the Annual Symposium on the Theory of Computing (STOC) 1998, pages 209–218. ACM Press, 1998.
- [Cor00] Jean-Sebastien Coron. *On the Exact Security of Full Domain Hash*. Advances in Cryptology — Crypto 2000, Volume 1880 of Lecture Notes in Computer Science, pages 229–235. Springer-Verlag, 2000.
- [Cor02] Jean-Sebastien Coron. *Optimal Security Proofs for PSS and Other Signature Schemes*. Advances in Cryptology — Eurocrypt 2002, Volume 2332 of Lecture Notes in Computer Science, pages 272–287. Springer-Verlag, 2002.
- [CP92] David Chaum and Torben Pedersen. *Wallet Databases with Observers*. Advances in Cryptology — Crypto’92, Volume 740 of Lecture Notes in Computer Science, pages 89–105. Springer-Verlag, 1992.
- [DDPY94] Alfredo De Santis, Giovanni Di Crescenzo, Giuseppe Persiano, and Moti Yung. *On Monotone Formula Closure of SZK*. Proceedings of the Annual Symposium on Foundations of Computer Science (FOCS)’94, pages 454–465. IEEE Computer Society Press, 1994.
- [DMPW98] Erik De Win, Serge Mister, Bart Preneel, and Michael Wiener. *On the Performance of Signature Schemes Based on Elliptic Curves*. Algorithmic Number Theory Symposium — ANTS-III, Volume 1423 of Lecture Notes in Computer Science, pages 252–266. Springer-Verlag, 1998.
- [DP92] Alfredo De Santis and Giuseppe Persiano. *Zero-Knowledge Proofs of Knowledge Without Interaction*. Proceedings of the Annual Symposium on Foundations of Computer Science (FOCS)’92, pages 427–436. IEEE Computer Society Press, 1992.
- [FS86] A. Fiat and A. Shamir. *How to Prove Yourself: Practical Solutions to Identification and Signature Schemes*. Advances in Cryptology — Crypto’86, Volume 263 of Lecture Notes in Computer Science, pages 186–194. Springer-Verlag, 1986.
- [GJ03] Eu-Jin Goh and Stanislaw Jarecki. *Signature Scheme as Secure as the Diffie-Hellman Problem*. Advances in Cryptology — Eurocrypt 2003, Vol-

- ume 2656 of Lecture Notes in Computer Science, pages 401–415. Springer-Verlag, 2003.
- [GQ88] Louis Guillou and Jean-Jacques Quisquater. *A Practical Zero-Knowledge Protocol Fitted to Security Microprocessor Minimizing Both Transmission and Memory*. Advances in Cryptology — Eurocrypt’88, Volume 330 of Lecture Notes in Computer Science, pages 123–128. Springer-Verlag, 1988.
 - [GT03] Shafi Goldwasser and Yael Tauman. *On the (In)security of the Fiat-Shamir Paradigm*. Proceedings of the Annual Symposium on Foundations of Computer Science (FOCS) 2003, pages 102–113. IEEE Computer Society Press, 2003.
 - [KW03] Jonothan Katz and Nan Wang. *Efficiency Improvement for Signature Schemes with Tight Security Reductions*. Proceedings of the Annual Conference on Computer and Communications Security (CCS). ACM Press, 2003.
 - [Mer88] R. Merkle. *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — Crypto’87, Volume 293 of Lecture Notes in Computer Science, pages 369–378. Springer-Verlag, 1988.
 - [MRH04] Ueli Maurer, Renato Renner, and Clemens Holenstein. *Indifferentiability, Impossibility Results on Reductions, and Applications to the Random Oracle Methodology*. Theory of Cryptography Conference (TCC) 2004, Volume 2951 of Lecture Notes in Computer Science, pages 21–39. Springer-Verlag, 2004.
 - [MV03] Daniele Micciancio and Salil Vadhan. *Statistical Zero-Knowledge Proofs with Efficient Provers: Lattice Problems and More*. Advances in Cryptology — Crypto 2003, Volume 2729 of Lecture Notes in Computer Science, pages 282–298. Springer-Verlag, 2003.
 - [Oka92] T. Okamoto. *Provable Secure and Practical Identification Schemes and Corresponding Signature Schemes*. Advances in Cryptology — Crypto’92, Volume 740 of Lecture Notes in Computer Science, pages 31–53. Springer-Verlag, 1992.
 - [Pas03] Rafael Pass. *On Deniability in the Common Reference String and Random Oracle Model*. Advances in Cryptology — Crypto 2003, Volume 2729 of Lecture Notes in Computer Science, pages 316–337. Springer-Verlag, 2003.
 - [PS00] David Pointcheval and Jacques Stern. *Security Arguments for Digital Signatures and Blind Signatures*. *Journal of Cryptology*, 13(3):361–396, 2000.
 - [Sch91] C.P. Schnorr. *Efficient Signature Generation by Smart Cards*. *Journal of Cryptology*, 4:161–174, 1991.
 - [SG02] Victor Shoup and Rosario Gennaro. *Securing Threshold Cryptosystems against Chosen Ciphertext Attack*. *Journal of Cryptology*, 15(2):75–96, 2002.

A Special Zero-Knowledge for Bounded Challenge Size

Lemma 1. *Every Fiat-Shamir proof of knowledge (P, V) with challenge size $\ell(k) = O(\log k)$ is special zero-knowledge.*

Proof. Let Z be a (regular) zero-knowledge simulator of the protocol and let $L = L(k) = 2^{\ell(k)} = \text{poly}(k)$. Then we construct a special zero-knowledge simulator Z_{sp} as follows (we only deal with the case YES, the case NO is trivial):

- Algorithm Z_{sp} gets x, ch, YES as input.
- Z_{sp} invokes kL independent copies of $Z(x, \text{YES})$ to generate kL transcripts $(\text{com}_i, \text{ch}_i, \text{resp}_i)$.
- Z_{sp} outputs the transcript $(\text{com}_i, \text{ch}_i, \text{resp}_i)$ with the smallest index i such that $\text{ch}_i = \text{ch}$; if none exists then Z_{sp} outputs $\perp$ instead.

The idea is that the output of Z should be indistinguishable from transcripts generated between P and V . In particular, V picks a random challenge ch and we thus expect the challenges output by Z to be sufficiently close to uniform. But then we will find some output $\text{ch} = \text{ch}_i$ with overwhelming probability among the kL samples. We next formalize this intuition.

First, it follows easily by a standard hybrid argument that the kL transcripts produced by Z are indistinguishable from kL independent executions of $(P(x, w), V(x))$. We conclude that the probability of Z_{sp} finding at least one transcript $(\text{com}_i, \text{ch}_i, \text{resp}_i)$ matching the given ch , i.e., with $\text{ch}_i = \text{ch}$, is overwhelming. Else it would be straightforward to distinguish the kL outputs of Z from ones generated by $(P(x, w), V(x))$. That is, for the latter the challenges are picked at random and the probability of covering all challenge values from $\{0, 1\}^{\ell(k)}$ is at least $1 - L \cdot (1 - \frac{1}{L})^{kL} \geq 1 - Le^{-k}$, which is overwhelming. A distinguisher could now test if all challenge values are hit, and would successfully distinguish the two cases if the probability for Z 's transcripts would not be overwhelming.

Under the condition that Z_{sp} does not return $\perp$ the output $(\text{com}, \text{ch}, \text{resp})$ of Z_{sp} is indistinguishable from a transcript $(\text{com}, \text{ch}, \text{resp}) \leftarrow (P(x, w), V^{\text{ch}}(x, \text{ch}))$. Otherwise it would again contradict the indistinguishability of the kL transcripts of $Z(x)$ and $(P(x, w), V(x))$. We discuss this in more detail next.

Assume that there exists a successful distinguisher $D = (D_0, D_1)$ for Z_{sp} 's output. Then we derive a distinguisher $D' = (D'_0, D'_1)$ for the kL executions of Z as follows. $D'_0(k)$ simulates $D_0(k)$ to get an output $(x, w, \text{ch}, \delta)$; we presume again for simplicity $(x, w) \in W_k$. Algorithm D'_0 outputs (x, w) and $\delta' = (\text{ch}, \delta)$. In the next stage D'_1 gets kL transcripts $(\text{com}_i, \text{ch}_i, \text{resp}_i)$, either produced by Z or by $(P(x, w), V(x))$. It picks the smallest index i such that $\text{ch}_i = \text{ch}$ (if none exists it stops with output 1), then it runs $D_1(\text{com}_i, \text{ch}_i, \text{resp}_i, \delta)$ and returns the final output of D_1 .

Note that D' only stops prematurely (if $\text{ch}_i \neq \text{ch}$ for all i) with negligible probability in either case. Under the condition that this does not happen, the output of D'_1 for transcripts by Z has the same distribution as the output of D_1 for a transcript generated by Z_{sp} . Similarly, given D' does not abort early, the output of D'_1 for transcripts by $(P(x, w), V(x))$ is identically distributed to the output of D_1 for a transcript produced by $(P(x, w), V^{\text{ch}}(x, \text{ch}))$. The claim now follows. $\square$

B Deterministic Verifiers in the Random Oracle Model

Lemma 2. *For every non-interactive Fiat-Shamir proof of knowledge (P, V) in the random oracle model, there exist a deterministic algorithm V' such that (P, V') is also a non-interactive Fiat-Shamir proof of knowledge for the same relation.*

Proof. Suppose $V^H(x, \pi)$ in the original protocol is probabilistic and uses random coins ω of polynomially length. By a standard trick we can split oracle H into two “independent” oracles H_π and H_ω (e.g., $H_\pi(\cdot) = H(0, \cdot)$ and $H_\omega(\cdot) = H(1, \cdot)$) and let the prover use H_π to produce the proof, while V^H uses $\omega = H_\omega(x, \pi)$ as the random coins to verify the proof π for x (with respect to oracle H_π). The extractor now only sees all queries to oracle H_π .

Completeness and zero-knowledge are clearly preserved. For extraction note that a malicious prover A has also access to oracle H_ω through H . We show that this cannot increase the success probability significantly, though. Denote by Succ_A the event that A manages to find (x, π) such that the transformed $V^H(x, \pi)$ accepts but $K(x, \pi, \mathcal{Q}_H(A))$ fails to return a valid witness w . Suppose that the probability of event Succ_A is noticeable. From A we construct an algorithm A' which contradicts the extraction property of the original protocol.

Let A make at most q many queries to H_ω . We can assume that A always asks H_ω about the final output (x, π) at some point, else we could make this additional query, increasing the total number only by one. We also assume that all queries are distinct, and therefore all answers are independent. Our derived algorithm A' with oracle H initially picks an index j between 1 and q and starts an emulation of A for oracles $H_\pi = H$ (relaying communication) and by providing consistent random answers on behalf of H_ω . If A forwards the j -th query (x, π) to H_ω then A' outputs (x, π) and stops.

Let pick_i be the event that the i -th query is output by A as a candidate. Then, for some i , $\text{Prob}[\text{Succ}_A \wedge \text{pick}_i]$ is at least $\text{Prob}[\text{Succ}_A]/q$. And the guess j of A' equals this index i with probability at least $1/q$. In this case, the random choice of ω_i corresponds to a run of the probabilistic verifier on input (x_i, π_i) with independent random coins. Moreover, A' only makes queries to oracle $H_\pi = H$, all answers for H_ω are chosen internally, $\mathcal{Q}_{(H_\pi, H_\omega)}(A) = \mathcal{Q}_H(A')$ and the extractor has the same probability for finding a witness. This, however, would contradict the negligible extraction error of the original protocol. $\square$